

Data analysis of gaseous Active Target detector

In this extended report, I will describe in more detail the work I carried out during the GSI/FAIR Summer Student Program 2026. The ACTAF (ACtive TArget for FAIR) detector is a gas-filled detector that operates simultaneously as an active target and a time projection chamber (TPC). The gas serves both as the target material and as the detection medium. During the program, I worked on the analysis of data mostly obtained during July 2026. My work focused on three main topics:

1. Charge collection dependence on changing HV settings
2. Noise analysis using Fast Fourier Transform (FFT)
3. Development of a summary table for gas quality monitoring using alpha sources

The following sections describe the experimental setup, the data used for the analysis, and the methods applied to investigate these three topics.

Experimental setup and Geometry

The ACTAF (ACtive TArget for FAIR) detector (see Fig. 1) is a gas-filled detector designed to operate both as an active target and as a time projection chamber (TPC). In an active-target configuration, the gas serves simultaneously as the target material and as the detection medium. This allows the reaction products to be detected directly inside the target volume.

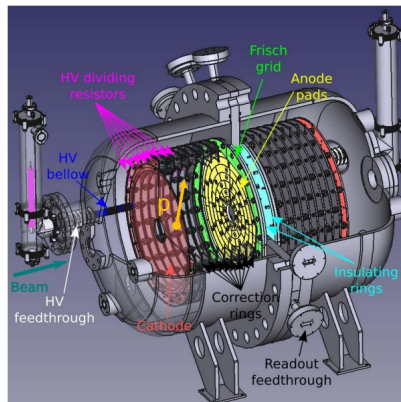


Figure 1: ACTAF TPC.

ACTAF is currently operated as part of the AMBER experiment at CERN. The detector was developed for experiments with radioactive beams by R3B collaboration at FAIR and AMBER at CERN. In the AMBER setup, ACTAF is used to detect charged particles produced in nuclear reactions and to provide information about their position and energy.

The active volume of the detector is read out by two anodes. Each anode contains 64 individual readout pads, giving a total of 128 pads. The pads are arranged in concentric rings around the centre of the detector. This geometry (see Fig. 2) allows the position of the detected signal to be determined from the pad on which it is recorded. In the 2026 detector configuration, five ^{241}Am alpha sources were installed inside the detector drift volume. These sources provide well-defined alpha-particle signals and are used for detector monitoring, calibration, and studies of the detector response under different operating conditions.

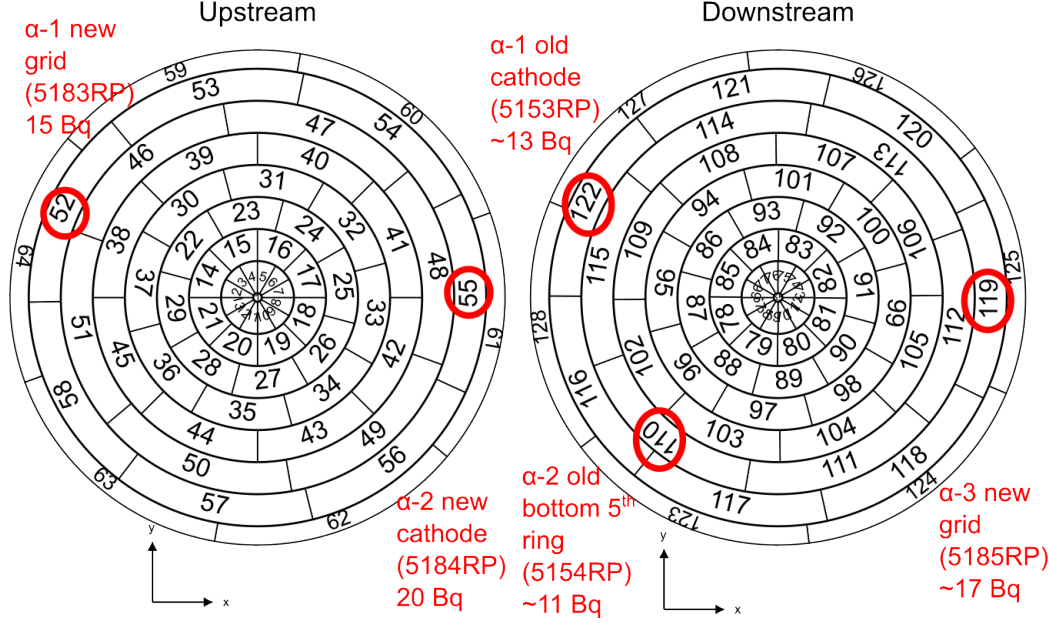


Figure 2: Geometry of anodes with marked positions of alpha sources.

The two anodes provide information from opposite sides of the detector, which is also used in the reconstruction and analysis of the recorded signals.

The geometry and pad numbering are important for the data analysis because each recorded waveform has to be assigned to the correct detector pad. The pad information is therefore used to identify the position of the signal and to analyse the detector response separately for individual pads.

Data processing chain

This section briefly introduces the types of data that are part of the R3BRoot data processing framework for ACTAF. There are three main data types. The first is called Mapped Data (*R3BActafMappedData*), which consists of the raw waveform together with the pad ID, so that each trace is associated with a specific pad — this pad assignment, along with the baseline determination, baseline subtraction, and integral calculation, is performed by *R3BActafReader* during unpacking. The resulting per-pad integral (Eraw) is stored directly in the Mapped Data and later histogrammed, without any recomputation, by *R3BActafOnlineSpectra*.

The second type of data is called Calibrated (Cal) Data (*R3BActafCalData*). Between Mapped Data and Cal Data, a signal-processing chain is applied in *R3BActafMapped2Cal*: the waveform is (optionally) smoothed using a Savitzky-Golay filter (*R3BActafUtils::ApplySGFilter*), the baseline is then determined (*R3BActafUtils::ComputeBaselineMean*), subtracted from the signal, and the integral is calculated (*R3BActafUtils::IntegratePulse*; at the time of writing this report, the integral is taken only up to the first zero crossing). The resulting integral is multiplied by a per-pad gain factor stored in *R3BActafCalPar* (at the time of writing this report the gain matching is not implemented yet and is being worked on by Teresa Aboal.) to obtain the calibrated energy.

The third and last type of data is Hit Data (*R3BActafHitData*). Between Cal Data and Hit Data, a position reconstruction step is performed in *R3BActafCal2Hit*: the (x,y) coordinates are obtained from the pad geometry (*R3BActafGeometry::GetPosition*), while z and the energy are carried over unchanged from Cal Data — z having been derived earlier from the drift time of the signal, and the energy from the calibrated pulse integral. Hit Data therefore contains the reconstructed position (x,y,z) together with the corresponding energy.

Following Hit Data, the hits are grouped into clusters corresponding to individual particle tracks in *R3BActafHit2Cluster*. Within each cluster, the calibrated energies of the belonging hits are summed to obtain

the total energy deposited by the particle, and the track angles are determined from an energy-weighted linear fit to the spatial positions of the clustered hits (*R3BActafHit2Cluster::FitTrack*).

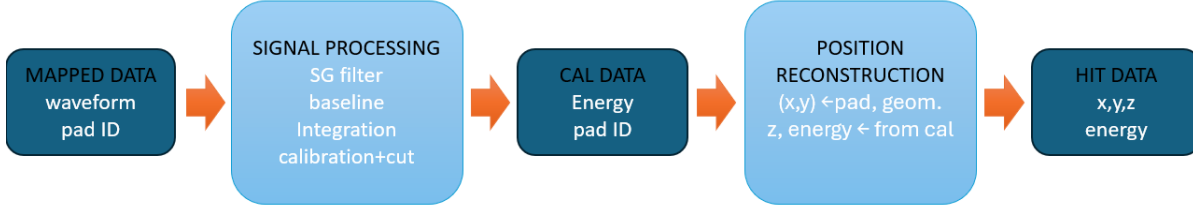


Figure 3: Brief scheme of Data processing chain.

Identification of Pads with visible alpha peak

The expected positions of the ^{241}Am alpha sources were known from the detector geometry. For these pads, the pulse-integral spectra were examined to identify the alpha peak.

Due to the relatively low activity of the ^{241}Am sources, the alpha peak was not clearly visible in every expected pad. In some cases, the signal was observed on a neighbouring pad, while in other cases no clear peak could be identified.

Therefore, each expected pad was checked individually and only pads with a clearly visible alpha peak were selected for further analysis. These selected pads were then used for the peak-fitting procedure and for studying the dependence of the alpha-peak position on the applied high voltage.

For checking individual pads I use macro called *alphaPeakAnalysis.cpp*. The macro *alphaPeakAnalysis* processes a single run stored as a ROOT tree and builds (ROOT file processed with *nearline* macro), for a predefined list of pads, both a one-dimensional spectrum of the baseline-subtracted trace integral and a two-dimensional histogram of that integral against the event number, which is useful for spotting drifts or instabilities over the course of the run. The run is opened with *TFile::Open(filename, "READ")* and the event tree *evt* is retrieved from it. The per-event trace data is stored in the branch *ActafMappedData*, from which the macro extracts two leaves through the helper function *GetLeafFromSubLocal()*: *ActafMappedData.fPad*, giving the pad number associated with each entry in the event, and *ActafMappedData.fTrace[2500]*, holding the raw 2500-sample trace for that entry. These are located once, before the event loop, rather than being looked up again for every event.

For every pad hit within an event, the trace is integrated by the function *IntegrateTraceLocal()*, which reproduces the logic of the real *IntegrateTrace()* used elsewhere in the analysis chain. The baseline is estimated as the mean of the samples from *startBackground* (sample 1500) to the end of the trace, and is then subtracted sample by sample while integrating from the first sample up to *endSignal* (sample 1100):

```

double sumBaseline = 0.0;
int nBaselineBins = 0;
for (int i = startBackground; i <= endBackground && i < nSamples; ++i)
{
    sumBaseline += traceLeaf->GetValue(offset + i);
    ++nBaselineBins;
}
double baselineMean = (nBaselineBins > 0) ? sumBaseline / nBaselineBins : 0.0;
double integral = 0.0;
for (int i = 0; i <= endSignal && i < nSamples; ++i)
    integral += (traceLeaf->GetValue(offset + i) - baselineMean);

```

Unlike some other implementations, this version does not restrict the sum to positive samples only; every baseline-subtracted sample in the signal window contributes to the integral, whether positive or negative.

Before an event is accepted into the histograms, its integral is checked by *PassAlphaThreshold()* against an amplitude window that depends on the pad's category:

```

if (category.Contains("grid", TString::kIgnoreCase))
    return integral >= gridLow && integral <= gridUp;
if (category.Contains("cathode", TString::kIgnoreCase))
    return integral >= cathodeLow && integral <= cathodeUp;
return true;

```

Pads categorized as "grid" or "cathode" must fall within their respective `fAlphaSourceGridLow/fAlphaSourceGridUp` or `fAlphaSourceCathodeLow/fAlphaSourceCathodeUp` bounds (currently 0 to 200000 raw ADC units for both, in *R3BActafOnlineSpectra.h* it is set in default from 40000 to 100000), while pads belonging to the "5th ring" category are passed through without any window cut, since the function returns true by default when neither category string matches.

The pads to be analyzed, together with their category and their expected or observed status, are declared as three parallel vectors near the top of the function:

```

std::vector<int> selectedPad =
    { 115, 122, 102, 103, 110, 117, 112, 119, 46, 51, 52, 48, 55 };
std::vector<TString> padCategory =
    { "Up grid", "Up grid", "Up cathode", ... };
std::vector<TString> padStatus =
    { "Expected & Observed", "Expected & Observed", "Expected", ... };

```

These three vectors are index-aligned, so `selectedPad[i]`, `padCategory[i]`, and `padStatus[i]` always describe the same pad; this is also where new pads are added or removed from the analysis. For each pad in the list, the macro creates a 1D histogram (`h_alpha_pad<pad>`, 2500 bins from 0 to 2500, in units of 10^3 ADC channels) and a 2D histogram (`h2_alpha_pad<pad>_vs_event`, 2000 bins in event number against the same integral axis), both with titles built dynamically from the pad's category, number, and status string.

The number of events actually processed is capped by the `maxEventsToProcess` argument (100000 by default, can be changed by root -l 'alphaPeakAnalysis("filename", "max number of events")') whenever it is smaller than the total number of entries in the tree; otherwise the whole tree is processed. The main loop then iterates event by event, and for each event over all pads present in that event (`mapPadLeaf->GetLen()`), comparing the current pad number against every entry of `selectedPad`. When a match is found, the trace is integrated, converted to units of 10^3 ADC channels (`integral_k = integral / 1000.0`), checked against the amplitude window, and, if it passes and is positive, filled into both the corresponding 1D and 2D histograms. A progress counter is printed to the console every 1000 events.

After the loop finishes, the input file is closed and the results are written to a new ROOT file named `alpha_histograms_<run>.root`, where `<run>` is derived from the input file name by `GetRunName()`, stripping the directory path and the `.root` extension. Besides the histograms themselves, this output file also stores a `TNamed` object summarizing the input file name and the number of processed events.

Alongside the ROOT file, the macro also generates a companion analysis macro, `alpha_histograms_<run>.C`, written out with plain `std::ofstream` calls. This generated macro, when run separately, reopens the histogram file, retrieves every 1D and 2D histogram by name, and draws them on two separate canvases arranged in a 4×4 grid, one canvas for the integral spectra (`Draw("HIST")`) and one for the integral-versus-event-number maps (`Draw("COLZ")`), which lets the pad spectra be inspected again later without having to re-run the full event loop.

Finally, the macro prints a short summary to the console, listing the number of entries collected in each pad's 1D histogram, and releases the histogram objects before returning.

To open output ROOT file from *alphaPeakanalysis.cpp* I used *openAlphaHistograms.cpp*. Rather than reprocessing the raw event tree, it simply reopens the ROOT file of pre-built histograms produced by that earlier macro, restricts them to a common amplitude window for visual inspection, and exports the resulting canvases as PNG images.

The file given as filename is opened read-only with `TFile::Open`, and the same list of pads used during the original histogram production is repeated here so that the correct histogram names can be reconstructed:

```

std::vector<int> selectedPad =
    { 115, 122, 102, 103, 110, 117, 112, 119, 46, 51, 52, 48, 55 };

```

Since this vector is not read back from the file itself, it must stay in sync with the selectedPad list used when the histograms were originally created; if a pad is added to or removed from one macro, the other has to be updated to match, or the corresponding histogram will simply be reported as missing.

A single display window, xLow to xUp (30 to 100, in units of 10^3 ADC channels, can be easily changed in the script), is defined and applied uniformly to every pad, unlike the amplitude cut used during histogram filling in `alphaPeakAnalysis`, which was category-dependent (grid versus cathode). Here the window only narrows the axis range for plotting and does not discard any underlying histogram entries.

The output file names are derived from the input file name by stripping the `.root` extension:

```
TString baseName = filename;
baseName.ReplaceAll(".root", "");
```

The macro then proceeds in two similar blocks. In the first, it retrieves each 1D integral histogram, `h_alpha_pad<pad>`, from the file, restricts its x-axis to the xLow–xUp window with `GetXaxis()->SetRangeUser(...)`, and draws it into one cell of a 4×4-divided canvas (c1), using the "HIST" draw option (see Fig. 4). Once all thirteen pads have been placed, the canvas is saved as `<baseName>_spectra.png`. In the second block, the same procedure is repeated for the 2D histograms (see Fig. 5), `h2_alpha_pad<pad>_vs_event`, this time restricting the y-axis (which carries the integral) to the same window and drawing each histogram with the "COLZ" option on a second 4×4 canvas (c2), saved as `<baseName>_vs_event.png`. In both blocks, a histogram that cannot be found in the file is reported to the console and simply skipped, leaving the corresponding canvas pad empty rather than stopping the macro.

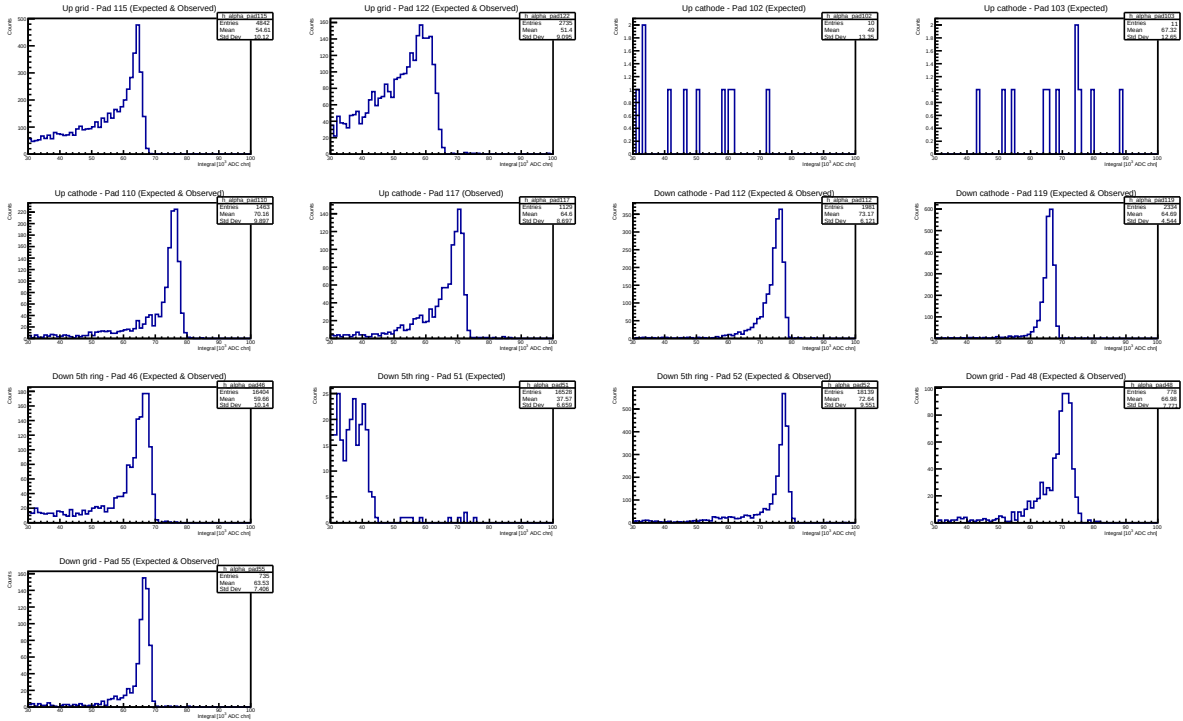


Figure 4: Example of 1D integral histograms done by `openAlphaHistograms.cpp`.

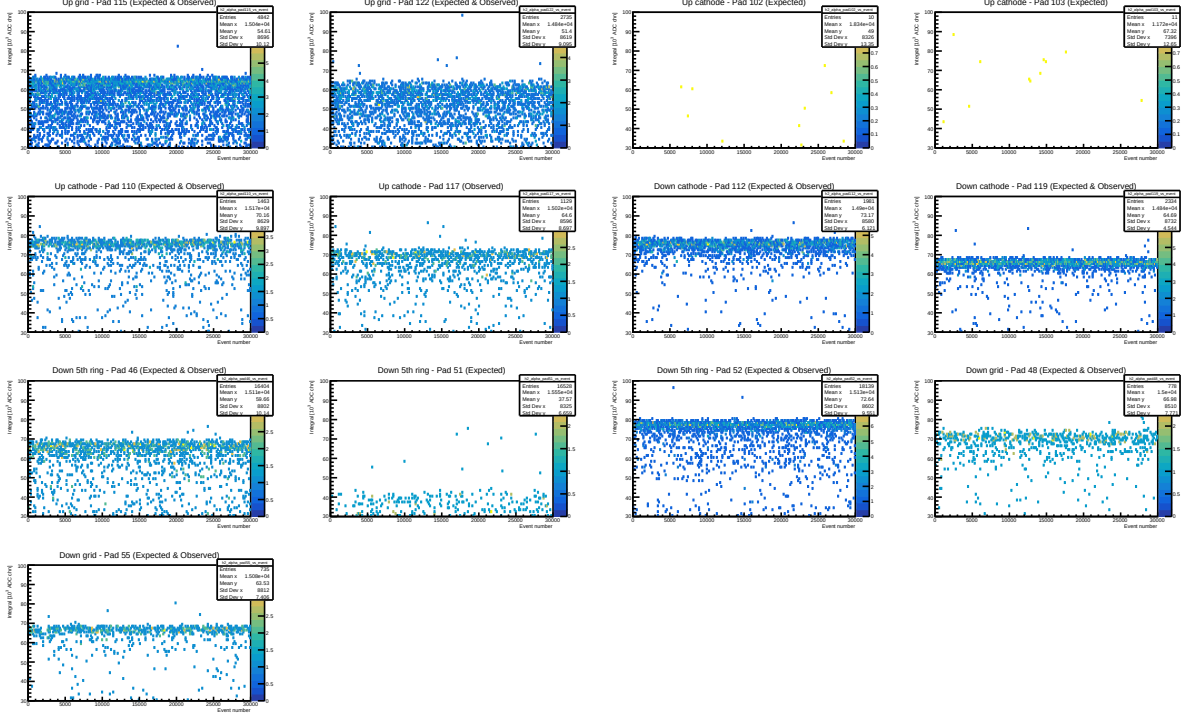


Figure 5: Example of 2D histograms done by *openAlphaHistograms.cpp*.

At that time this analysis was performed for runs 4122-4131 (see Fig. 6). The results were summarized in Table 1. Only the pads with clearly visible alpha peaks were selected for the next part of the analysis.

Table 1: Expected and observed pad positions for ^{241}Am alpha peaks in the 2026 detector configuration.

Source	Expected	Observed
Down cathode	115, 122	115, 122
Down 5th ring	102, 103, 110	110, 117
Down grid	112, 119	112, 119
Up grid	46, 51, 52	46, 52
Up cathode	48, 55	48, 55

Charge collection dependence upon HV

During July 2026, a few dedicated runs (see Fig.6) were performed with varying high voltage (HV) from 90% to 5% of the nominal value. This HV scan was performed to verify that the charge collection behaves as expected.

For these measurements, the ^{241}Am alpha sources inside the detector were used to provide a well-defined signal. The response of the detector was studied by looking at the pulse integral of the recorded alpha signals. The position of the alpha peak was then compared for different HV settings.

```

We recorded few more runs today at different HV and 4 bar:
4122 TPC HV scan with alpha: 18kV & 1350V: 90%nominal
4123 TPC HV scan with alpha: 16kV & 1200V: 80%nominal
4124 TPC HV scan with alpha: 14kV & 1050V: 70%nominal
4125 TPC HV scan with alpha: 12kV & 900V: 60%nominal
4126 TPC HV scan with alpha: 10kV & 750V: 50%nominal
4127 TPC HV scan with alpha: 8kV & 600V: 40%nominal
4128 TPC HV scan with alpha: 6kV & 450V: 30%nominal
4129 TPC HV scan with alpha: 4kV & 300V: 20%nominal
4130 TPC HV scan with alpha: 2kV & 150V: 10%nominal
4131 TPC HV scan with alpha: 1kV & 75V: 5%nominal

```

Figure 6: Runs used for HV scan.

Run 4035 with 4 bar pressure and nominal 100% was added to the analysis to compare other runs with nominal conditions.

The macro *plot_pad.cpp* automates the processing of the detector response of a single pad across a runs taken at different HV settings. For each run, the script loads the corresponding histogram, locates and fits the alpha peak, and subsequently uses the fit results to build the dependence of the peak position (Mean) and of the peak integral on the applied HV. The input consists of one ROOT file per run. The list of file names, the corresponding run numbers, and the associated HV values are defined at the beginning of the *plot_pad()* function:

```

std::vector<std::string> fileNames = {
    "online_data_4035_20260909_122322.root",
    "online_data_4122_20260831_170741.root",
    ...
};
std::vector<std::string> runNumbers = {"4035", "4122", "4123", ...};
std::vector<double> HV = {100, 90, 80, 70, 60, 50, 40, 30, 20, 10};

```

This is the part of the script that needs to be edited whenever a new run is added or a different data set is processed. The three vectors — *fileNames*, *runNumbers*, and *HV* — must have the same length and must be index-aligned, so that *fileNames[i]*, *runNumbers[i]*, and *HV[i]* always refer to the same run. The HV values, expressed as a percentage of the nominal voltage, form the x-axis of the resulting dependency plots.

Since the latest revision, the pad number is no longer hardcoded in the body of the script but is instead passed in as a function argument with a default value:

```

void plot_pad(int pad = 115) { ... }

```

The macro can therefore be run either without an argument, in which case the default pad 115 is processed, or with an explicit pad number, e.g. *root -l -q 'plot_pad.cpp(122)'*.

The alpha peak is fitted with a Gaussian sitting on top of a linear background, i.e. the compound function *pol1 + gaus*. This is applied uniformly to every pad, and is controlled by the boolean flag *useCompoundFit*:

```

bool useCompoundFit = true;

```

Because the flag is fixed to true, the branch of the main fit that builds *pol1(0) + gaus(2)* is the one that always executes, regardless of which pad number is passed to the macro. The plain-Gaussian branch is still present in the code as an else case, but with *useCompoundFit* hardcoded this way it is never reached.

For every run, indexed by *r* in *for (int r = 0; r < nRuns; r++)*, the script performs the following sequence of operations.

The histogram is first loaded from the corresponding ROOT file:

```

TFile *f = TFile::Open(fileNames[r].c_str());
TH1F *h = (TH1F*)f->Get(Form("fh1_Pad_%d_Eraw", pad));
h->SetDirectory(nullptr);
f->Close();

```

The raw-energy histogram (Eraw) for the selected pad is retrieved from the open ROOT file. Its name is built dynamically from the pad number (fh1_Pad_<pad>_Eraw), so if the naming convention of the histograms in the data changes, this format string is the place to update. Note for using fh1_Pad_<pad>_Eraw histogram for alpha peak analysis the range need to be changes from (0,40000) to (0,200000) otherwise the alpha peak will not be visible because is around 50000 ADC Channels. Other possibility is to use dedicated histogram for alpha peaks called fh1_alphaenergy, but with this histograms the algorithm for fitting needs to be adjusted because double peaks will be visible in few of these histograms. (The computing of the baseline for Eraw histograms and alphaenergy histograms differs a bit see for alphaenegy see IntegrateTrace in *R3BActafOnlineSpectra.cxx*, but there is no difference on the exponential behavior of mean alpha peak vs % of nominal HV disscusted later in results).

The alpha peak is then located using TSpectrum within a fixed ADC-channel window:

```
h->GetXaxis()->SetRangeUser(30000, 70000);
TSpectrum *s = new TSpectrum(5);
int nfound = s->Search(h, 15, "", 0.2);
```

The search is restricted to the range 30000–70000 ADC channels, and the parameters of Search() — a resolution of 15 channels and a threshold of 0.2 — control the sensitivity of the peak finder. These values may need to be adjusted if the peak is not identified correctly for a given data set.

Because the runs are ordered by decreasing HV and the peak position shifts smoothly rather than abruptly as the voltage is lowered, the correct candidate among those returned by TSpectrum is selected using the peak position found in the previous successful run as an anchor:

```
if (prevPeakPos > 0) {
    alphaPeakPos = *std::min_element(candidates.begin(), candidates.end(),
        [prevPeakPos](double a, double b) {
            return std::abs(a - prevPeakPos) < std::abs(b - prevPeakPos);
        });
    if (std::abs(alphaPeakPos - prevPeakPos) > maxJumpFromPrev) {
        alphaPeakPos = prevPeakPos;
    }
} else {
    std::sort(candidates.rbegin(), candidates.rend());
    alphaPeakPos = candidates[0];
}
```

The candidate closest to the previous peak position is chosen, which prevents the algorithm from mistaking a noise bump for the actual peak at low HV.

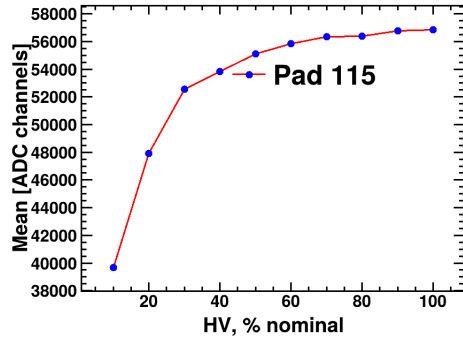
A preliminary Gaussian fit around alphaPeakPos, within a window of initWindow (2500 channels), then gives a first estimate of the mean and sigma:

```
TF1 *preFit = new TF1(..., "gaus", alphaPeakPos - initWindow, alphaPeakPos + initWindow);
h->Fit(preFit, "RQ0");
```

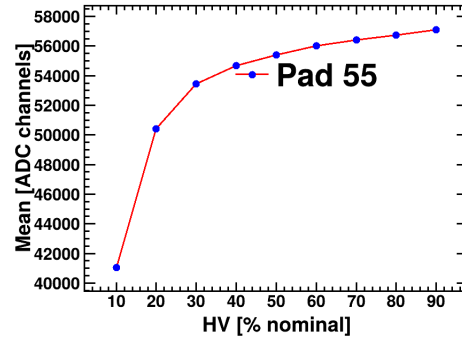
These estimates set the window for the main fit, pol1(0) + gaus(2), whose resulting mean and sigma are used for the rest of the analysis. The peak integral is then computed as the summed bin content within $\pm 3\sigma$ of the mean (TH1F::Integral, without the "width" option), giving the number of counts in that region.

For each run, the run number, HV, mean, sigma, ROI boundaries, and integral are written to both the console and fit_results_pad_<pad>.txt. Once all runs are processed, the script saves a 5×2 canvas of all histograms and fits (Pad_<pad>_AllRuns_Fits.png) together with the two dependency plots, Pad_<pad>_Mean_vs_HV.png and Pad_<pad>_Integral_vs_HV.png, showing the fitted peak position and count integral as functions of HV.

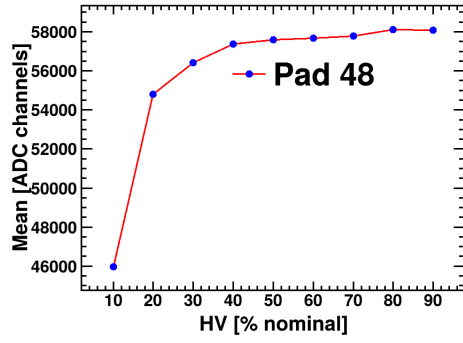
Results of this analysis



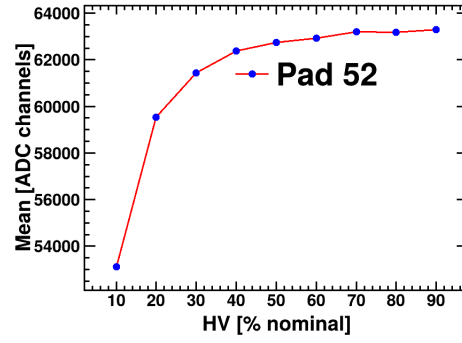
a)



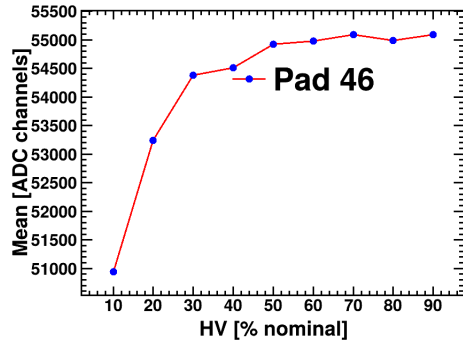
b)



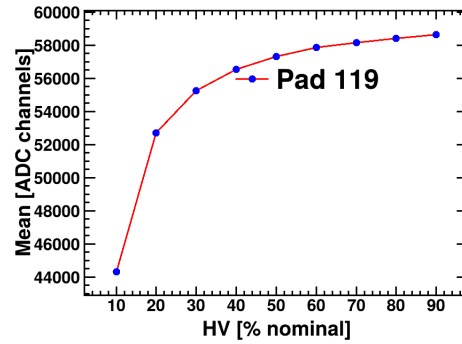
c)



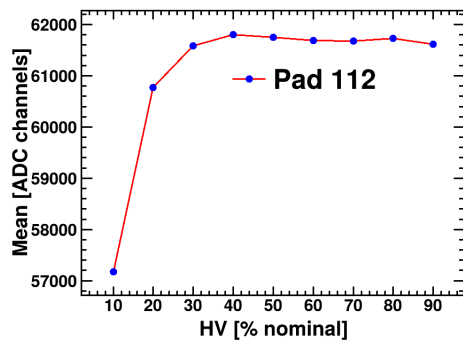
d)



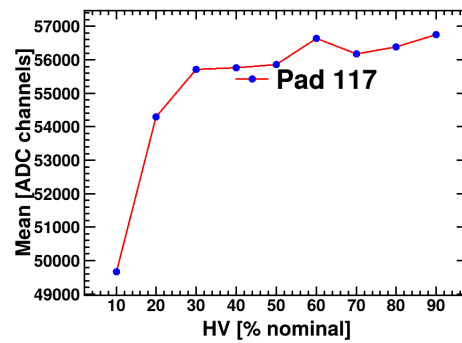
e)



f)



g)



h)

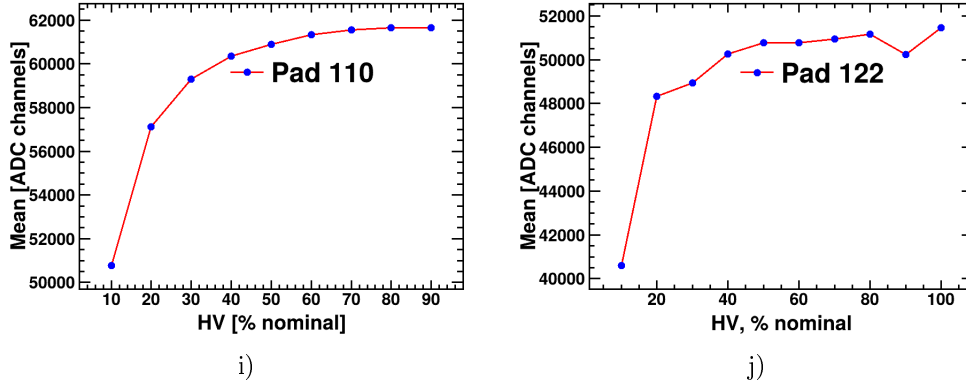


Figure 7: Mean vs HV.

Noise analysis using Fast Fourier Transform

During the analysis of the detector signals, electronic noise was observed in the recorded waveforms of several pads. In the time domain, the noise appeared as periodic fluctuations in the signal, but its frequency could not be determined directly from the waveform.

For this I wrote script called *waveform_Viewer.cpp*. The *waveform_Viewer* macro is an interactive ROOT tool used to inspect raw detector waveforms and verify pulse integration parameters. It works with an evt tree inside a ROOT file, searching for events from a specified pad where the signal exceeds chosen energy and height cutoffs. You run the macro directly inside ROOT or from the system command line by passing parameters into the main function call *waveform_Viewer(targetPad, integralThreshold, amplitudeThreshold, filename)*. The pad, threshold, and input file can be changed by modifying the function arguments when starting the script—for example, passing 122 for the target pad, 100.0 for the minimum integrated energy, 60.0 for the minimum amplitude in ADC channels, and the string path to your input ROOT file:

```
root -l 'waveform_Viewer.cpp(122, 100.0, 60.0, "/path/to/data.root")'
```

Once started, the script loops through all entries in the file, reads the raw waveform array *fTrace[2500]* for the chosen *targetPad*, and determines the baseline level by averaging the first 500 samples (in real data processing computing of baseline differs but for this application to roughly know the integral value it was enough) using *ComputeBaselineMean()*. It then searches for the maximum peak using *FindMaxPosition()* and determines the left and right pulse edges by walking outward until the signal drops back below the baseline using *IntegratePulseWithBounds()*. The net pulse integral is calculated by summing the baseline-subtracted ADC channels within these boundaries, and the event is stored only if *Integral > integralThreshold* and the peak height exceeds *amplitudeThreshold*. All passing events are then displayed sequentially on a *TCanvas*, showing the raw waveform as a *TH1F* histogram alongside a shaded green *TGraph* overlay indicating the exact integration window. Real-time text generated via *TLatex* displays the baseline, amplitude, calculated integral, and the value from the file's *fE* leaf. You navigate through the filtered events directly via terminal prompt commands: entering *n* moves to the next event, *p* goes back to the previous one, *j* prompts for an event index to jump to, *s* exports the plot to a *.png* image, and *q* terminates the viewer.

The analysis was performed using the Mapped Data waveforms from selected pads. The frequency spectra were calculated for the recorded events and then averaged for each run. The resulting spectra were used to identify the main frequency components of the electronic noise and to study their dependence on the applied high voltage.

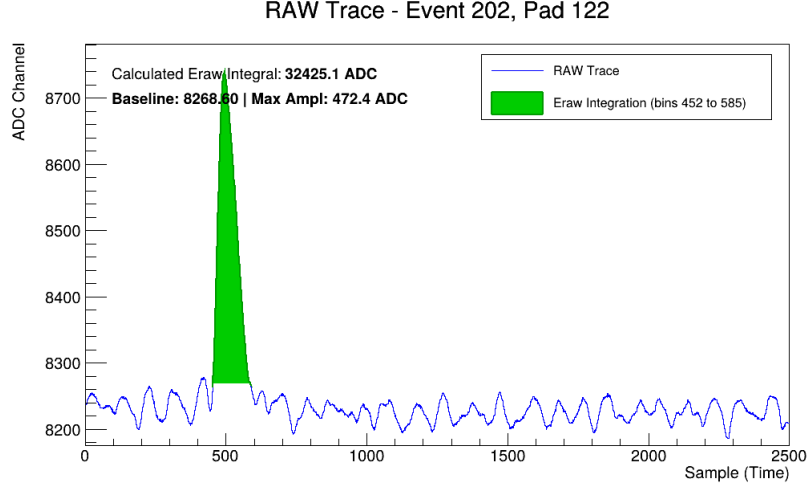


Figure 8: Example of Raw Waveform displayed with waveform_Viewer.cpp.

Sample can be converted to Time by multiplying Sample by 51.44 ns.

Fast Fourier Transformation was done with macro *average_FFT.cpp*.

The *average_FFT3* macro is a ROOT analysis utility used to calculate and visualize the average Fast Fourier Transform (FFT) power spectrum of detector signals, allowing users to identify dominant noise frequencies across multiple events. It processes raw waveform traces from the evt tree of a ROOT file, extracts the signals corresponding to a selected pad, subtracts the baseline, computes the power spectrum with a real-to-complex FFT (TVirtualFFT, FFTW backend), and saves the resulting averaged spectrum for subsequent fitting.

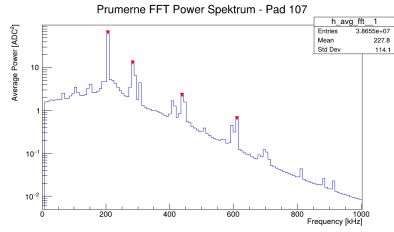
The macro can be executed directly in ROOT or from the command line by calling *average_FFT(targetPad, samplingRateHz, filename)*. To change the pad, the sampling frequency or the input file, simply adjust the function arguments, for example passing 55 for the target pad, 19.44e6 for the sampling rate in Hz, and the path to the input data file: `root -l 'average_FFT3.cpp(55, 19.44e6, "/path/to/data.root")'`. The sampling rate should not be changed: each module samples at 14-bit resolution using a 77.76 MHz clock distributed from the trigger system, and the digitized data are averaged, producing one mean value from every 4 consecutive samples. Data corresponding to an effective clock frequency of 19.44 MHz are written out event by event.

Upon execution, the macro loops through all entries in the tree and reads the raw trace array *fTrace[2500]* for the specified targetPad. For each matching event, it calculates the mean ADC value across all 2500 samples and subtracts it to remove the DC offset before applying the FFT. The power in frequency bin *k* is calculated as $P_k = 4|X_k|^2/N^2$, where $N = 2500$, i.e. the squared amplitude of the corresponding harmonic component in ADC². The resulting values are accumulated over all matching events and divided by their number to obtain the mean power spectrum. The spectrum is stored in a TH1F histogram with 1250 bins of width $f_s/N = 7.776$ kHz, spanning from 0 kHz up to the Nyquist frequency $f_{max} = f_s/2 = 9.72$ MHz, with each bin centred on its FFT frequency $k \cdot f_s/N$. No window function is applied. The average power spectrum is displayed on a log-scale TCanvas and automatically saved into an output ROOT file named *fft_result_pad<pad>_<input file name>.root* for further analysis and noise component fitting.

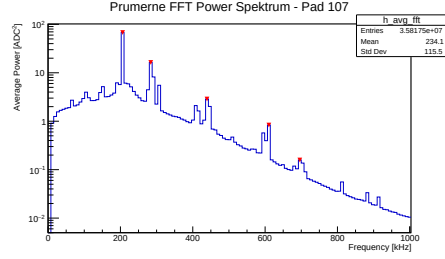
To analyse the averaged spectrum I wrote the macro *fitFFT.cpp*. It performs automated multi-peak detection and fitting on the averaged FFT power spectrum histogram stored in the ROOT file. The spectrum is displayed up to 1 MHz on a logarithmic scale, and a three-stage search using TSpectrum (the threshold and sigma parameters must be adjusted for every pad based on the plot, otherwise the correct peaks are not found) and local-maximum checks locates dominant, weak and intermediate noise peaks while removing duplicates. Each identified peak is fitted. The total error of the peak position is obtained by combining the statistical fit error and the bin-discretization uncertainty in quadrature. The fitted parameters are printed to the console and the canvas is exported as PDF and as an executable C file.

At first, a few pads were selected based on observation of the *E_raw* histogram produced by R3BActaOnlineSpectra, for example pad 107. Then more pads were selected based on viewing raw waveforms, for example pads 55

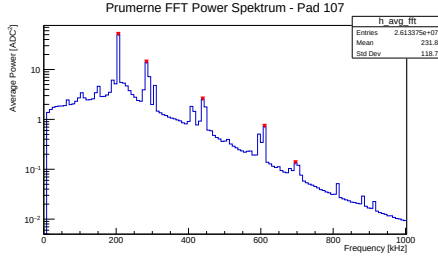
and 122. Finally, a few more pads were selected randomly to compare the results.
Results for Pad 107



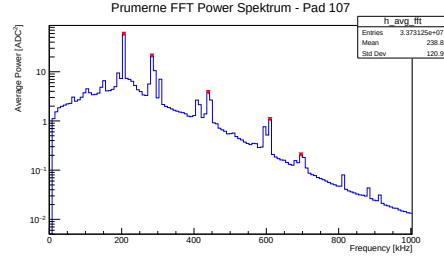
a) run 4122-90%



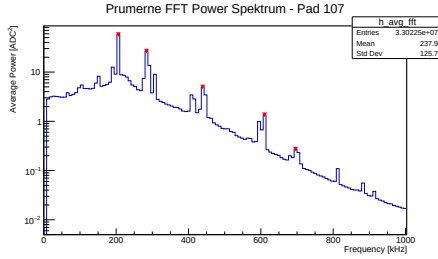
b) run 4123 80%



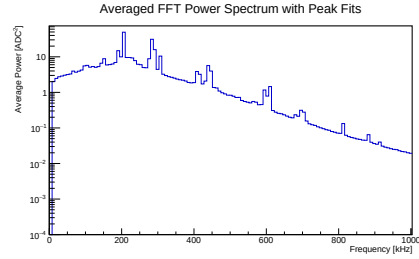
c) run 4124 70%



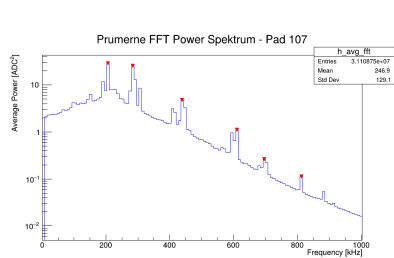
d) run 4125 60%



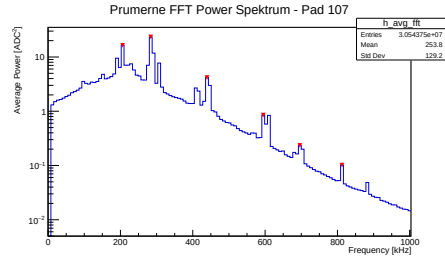
e) run 4126 50%



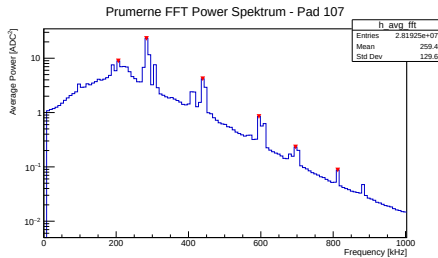
f) run 4127 40%



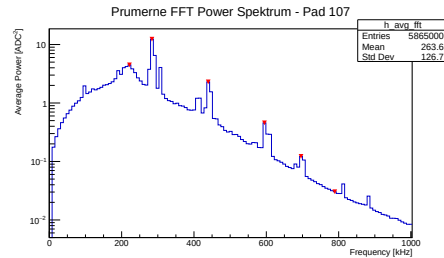
g) run 4128 30%



h) run 4129 20%



i) run 4130 10%



j) run 4131 5%

Every inspected pad showed frequency components at approximately 206 kHz, 280 kHz, and 610 kHz. Other peaks were also visible, but their frequencies differed between individual pads. The largest number of peaks was observed in the FFT spectrum of Pad 107. The reason for this behaviour is currently unknown, but this pad appears to pick up more noise than the other pads. This was also observed in the ERaw histogram done by *R3BActafOnlineSpectra*, where only a noise peak was visible. Therefore, Pad 107 was selected for a more detailed comparison with Pad 106, which was chosen as a representative pad because the noise observed on the other pads was similar to that of Pad 106.

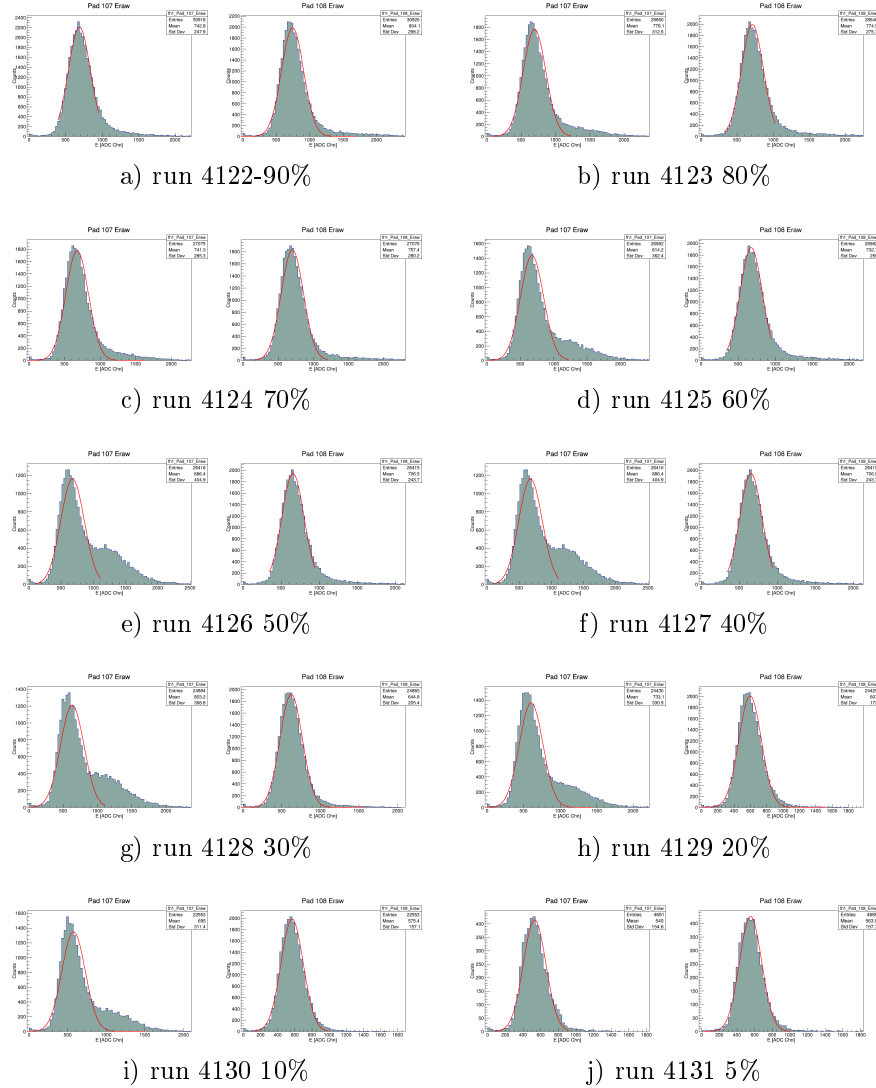
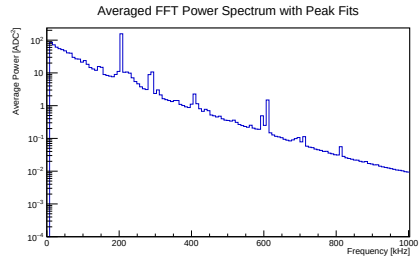
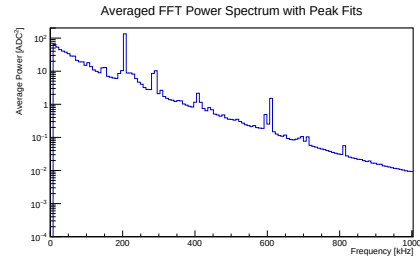


Figure 9: Comparison of noise in spectra for Pad 107 and Pad 106.

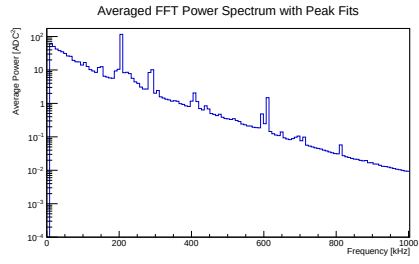
Pad 55



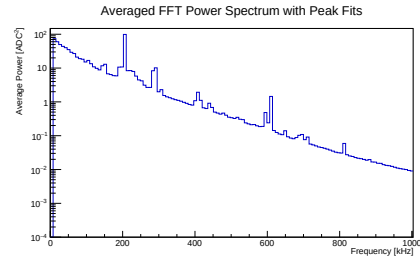
a) run 4035-100%



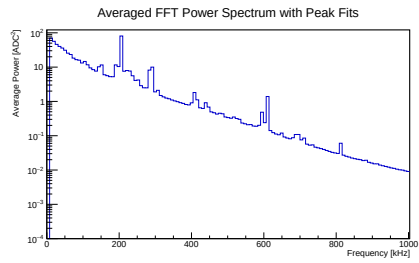
b) run 4122 90%



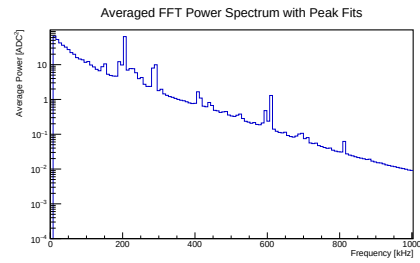
c) run 4123 80%



d) run 4124 70%



e) run 4125 60%



f) run 4126 50%

Pad 106

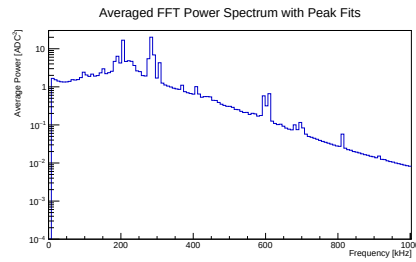


Figure 10: Pad 106 run 4131 10%.

Pad 98

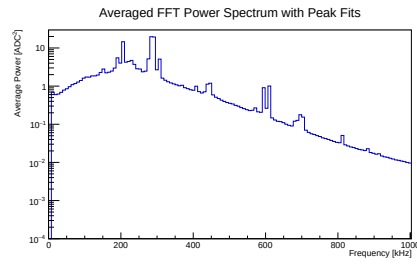
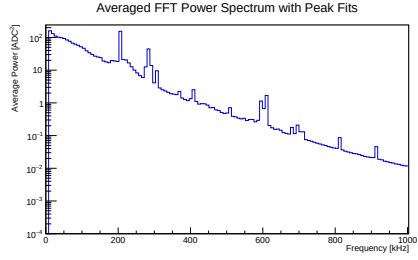
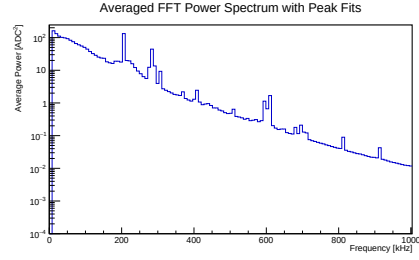


Figure 11: Pad 98 run 4131 10%.

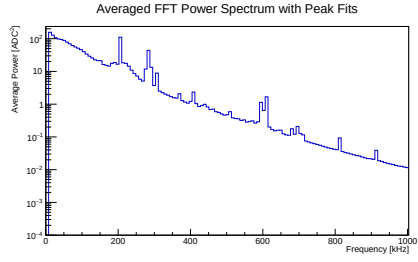
Another inspected pad 122.



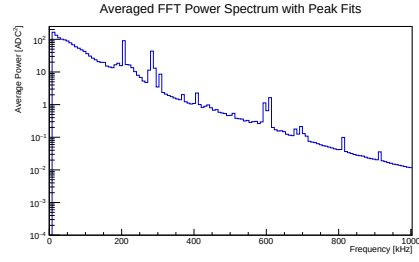
a) run 4122-90%



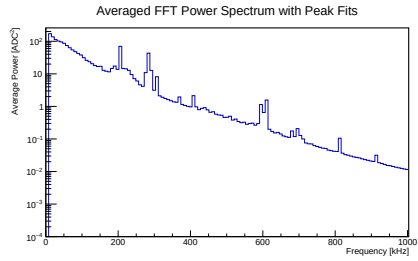
b) run 4123 80%



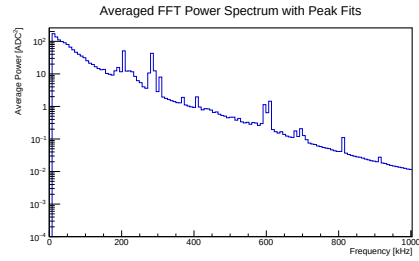
c) run 4124 70%



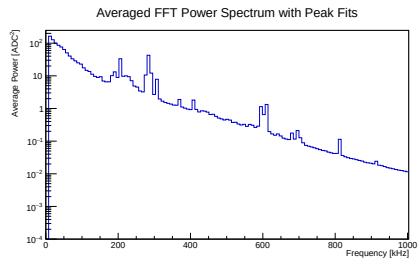
d) run 4125 60%



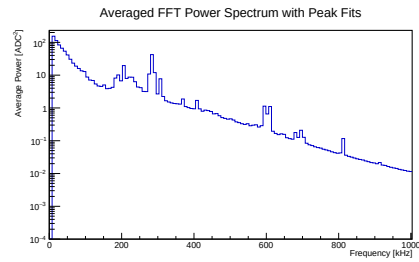
e) run 4126 50%



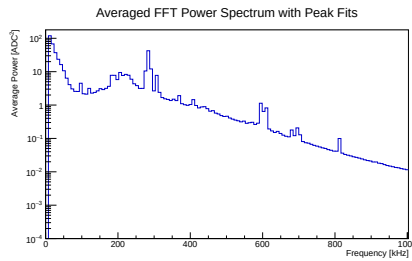
f) run 4127 40%



g) run 4128 30%



h) run 4129 20%



i) run 4130 10%

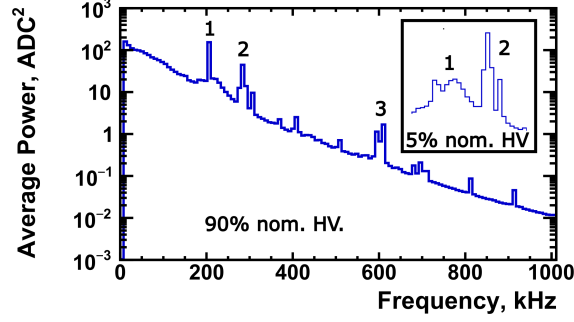


Figure 12: FFT Pad 122.

Since the 206 kHz component was observed on all inspected pads, I further investigated its dependence on the applied high voltage. The FFT analysis was performed for all available runs in which the HV was changed. The amplitude of the 206 kHz component decreased with decreasing HV. This behaviour suggests that the observed oscillation may be related to the high-voltage system or its power supply. However, the exact origin of the 206 kHz component cannot be confirmed from these measurements alone, and further measurements would be needed to determine whether the power supply is the source of this noise.

To investigate whether the observed noise components are also present after the signal processing, the FFT spectra from Mapped Data were compared with the corresponding Calibrated Data. The same frequency components were still visible in the Calibrated Data, including the component at approximately 206 kHz. This indicates that the observed noise is not fully removed during the calibration process.

In the Calibrated Data, the Savitzky-Golay filter is applied as part of the signal processing. However, the comparison shows that the main noise components are still visible after this filtering.

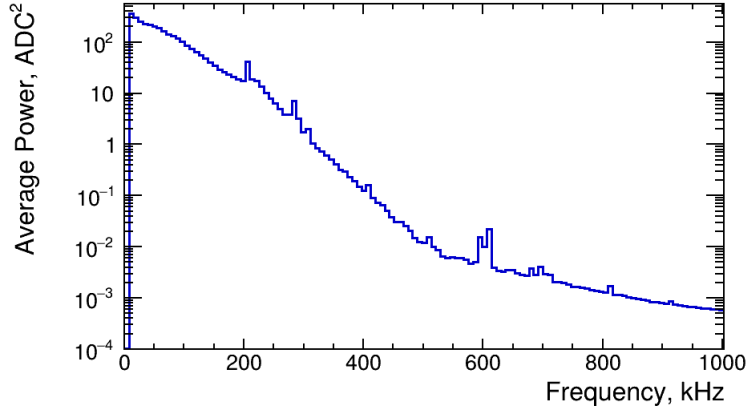


Figure 13: FFT for pad 122 for Cal data run 4122.

The FFT analysis showed that the electronic noise contains several frequency components, with the most pronounced components observed at approximately 206 kHz, 280 kHz, and 610 kHz. The 206 kHz component was present on all inspected pads and its amplitude decreased with decreasing HV. This dependence suggests a possible connection between the observed noise and the high-voltage system or its power supply. The same frequency components were also observed in the Calibrated Data, indicating that they are not fully suppressed by the applied signal processing. The exact origin of the noise remains unknown and would require further investigation.

Database for gas quality using alpha sources

The third part of my work focused on creating a summary table for monitoring the detector response using the installed ^{241}Am alpha sources. The purpose of the table is to provide a simple overview of the measured alpha-peak parameters for different measurements and HV settings.

For each measurement, the relevant parameters can be recorded, including the pad number, applied HV, alpha-peak position, and other parameters obtained from the peak fitting. This makes it possible to compare the detector response between different measurements and to monitor changes in the alpha-peak position over time.

The same script, *alphaPeakAnalysis.cpp*, which was used for the identification of pads with visible alpha peaks and explained in that section, was also used to obtain the alpha-peak parameters for the summary table. The script identifies and fits the visible ^{241}Am alpha peaks for the selected pads and HV settings. The obtained peak parameters were then collected in a table to provide an overview of the detector response for the different measurements. Then I used another macro to do the fitting. The fitting itself was done by the macro *fitAllPadsAlpha.cpp*. It takes the ROOT file of one run, which contains one histogram for each pad, and looks for the alpha peak in each of them. The peak is then fitted with a Crystal Ball function, which is a Gaussian with an additional tail on the low-signal side and is therefore suitable for peaks with a tail. For pad 112, where the background below the peak was significant, a linear background was added to the fit. For every pad where the fit was successful, the macro saves the peak position and width together with their uncertainties and the quality of the fit) into a text file, one file per run. The positions and widths are given in units of 10^3 ADC channels. The macro also saves images of all the fits, 16 pads per page, which makes it easy to check by eye that the fits look reasonable.

The text files from the fitting contain only the pad numbers and the fit results, without any information about the conditions of the measurement. For this reason I wrote another macro, *makeTable.C*, which connects the fit results with the experimental logbook. The important information from the logbook, such as the run number, pressure, HV, beam status and comments, is kept in a simple text file with one line per run. The macro takes the run number from the name of the fit-result file, finds the corresponding line in the logbook and creates a table with the columns pad, Mu, MuErr, Sigma and SigmaErr, where Mu and Sigma are the position and width of the alpha peak. The table has a header line with the conditions of the run, including the HV also as a percentage of the nominal value for the given pressure. The macro can be run either on a single file or on a whole folder, in which case the tables of all runs are written one after another into one text file, sorted by run number. When a new measurement is taken, it is enough to add one line to the logbook file. In this way, the table gives a simple overview of the alpha-peak parameters for the different measurements and HV settings.

To this table i have added only runs visible below (see Fig. 14)

Pressure, bar HV, kV Beam, y/n Run Comments	
We recorded few more runs today at different HV and 4 bar:	4 20 n 4093
4122 TPC HV scan with alpha: 18kV & 1350V: 90%nominal	4 20 y 4035
4123 TPC HV scan with alpha: 16kV & 1200V: 80%nominal	4 20 y 3886
4124 TPC HV scan with alpha: 14kV & 1050V: 70%nominal	4 20 n 3744 just after ramping HV up
4125 TPC HV scan with alpha: 12kV & 900V: 60%nominal	20 100 y 2898 cooling of TPC installed and running
4126 TPC HV scan with alpha: 10kV & 750V: 50%nominal	20 100 y 2888 before installation of the cooling of TPC
4127 TPC HV scan with alpha: 8kV & 600V: 40%nominal	20 95 y 2440 HV at 95%
4128 TPC HV scan with alpha: 6kV & 450V: 30%nominal	20 90 y 2424 HV at 90%
4129 TPC HV scan with alpha: 4kV & 300V: 20%nominal	20 90 n 2411 HV 90kV/6750V
4130 TPC HV scan with alpha: 2kV & 150V: 10%nominal	16 80 y 2378
4131 TPC HV scan with alpha: 1kV & 75V: 5%nominal	8 40 y 2363
	8 40 y 2362
	8 40 y 2341

Figure 14: Runs added into table.

The summary table is stored as a plain text file, which makes it easy to read, to edit and to keep under version control. It consists of one block for each run, and the blocks are sorted by the run number. Each

block starts with a header line containing the run number and the conditions of the measurement taken from the logbook (pressure, HV, the corresponding voltage, the HV as a percentage of the nominal value, the beam status and a comment), followed by the name of the fit-result file the numbers come from. Below the header there is a table with one row for each pad and the columns Pad, Mu, MuErr, Sigma and SigmaErr, where Mu and Sigma are the position and width of the alpha peak and MuErr and SigmaErr are their uncertainties from the fit. The number of entries and the χ^2/NDF of each fit are not included in the summary table, but they are kept in the original fit-result files. An example of one block for run 2362 is shown in Table 1. In the text file the table is drawn with ASCII characters, here it is redrawn as a normal table.

Only the pads with a successful fit are listed. If no alpha peak was found in a pad, if the peak was too low, or if the fit failed, the pad is simply skipped, so the number of rows can be different for each run. For runs where some of the conditions are not written in the logbook, only the available information is shown in the header. The complete table for all processed runs, together with the logbook, is available as text files in the GitHub repository (<https://github.com/Dolezal-Patrik/GSI-FAIR---Summer-school---ACTAF-Data-analysis>).

Run 4123				
P = 4 bar, HV = 16 kV, V = 1200 V				
80 % of nominal HV, beam: n				
TPC HV scan with alpha				
Pad	Mu	MuErr	Sigma	SigmaErr
46	67.029	0.160	1.266	0.102
48	71.312	0.177	1.498	0.126
51	37.480	1.048	3.231	0.909
52	77.595	0.055	1.290	0.062
55	66.547	0.096	1.282	0.104
110	76.017	0.124	1.338	0.097
112	76.178	0.080	1.222	0.079
115	64.588	0.085	1.193	0.054
117	70.506	0.221	1.230	0.126
119	66.267	0.049	1.126	0.041
122	61.424	0.236	1.589	0.137

In summary, the table created in this part of the work provides a simple and compact overview of the alpha-peak parameters measured with the installed ^{241}Am sources. For each run, it contains the position and width of the alpha peak for every pad where the peak could be fitted, together with the conditions of the measurement taken from the logbook. This makes it possible to compare the detector response between different runs and HV settings, and, when new measurements are added, to follow how the alpha-peak position changes with time. A significant shift of the peak position measured under the same pressure and HV conditions could be an indication of a change in the detector response and, potentially, in the gas conditions.

The table also has some limitations. It contains only the pads where an alpha peak is visible and where the fit was successful, so the set of pads can differ from run to run. The values are also meaningful only when they are compared for the same pressure and HV, since the position of the peak depends strongly on them. So far, only a few runs were added to the table, mainly the HV scan with alpha particles at 4 bar, and it will become more useful as more measurements are collected. New runs can be added easily, by adding one line to the logbook and running the two macros described above, and the resulting table can be used directly for further analysis, for example to plot the peak position as a function of HV or of the run number.

Conclusion

In this extended report, I have summarized the work I carried out during the GSI/FAIR Summer Student Program. My main tasks focused on analyzing the data from the ACTAF detector, specifically investigating the charge collection dependence on varying high-voltage settings, performing noise frequency analysis using Fast Fourier Transform (FFT), and creating an initial summary table for gas quality monitoring based on the installed ^{241}Am alpha sources. All scripts, macros, and summary tables developed during this project have been uploaded and are available in the GitHub repository (<https://github.com/Dolezal-Patrik/GSI-FAIR--Summer-school---ACTAF-Data-analysis>). The raw detector data and large ROOT data files are stored on the GSI cluster/storage infrastructure (Lustre file system). I hope that the scripts, workflows, and findings presented in this report will be useful for future data analysis, detector monitoring, and further development of the ACTAF detector within the R3B/AMBER collaborations.

References

- [1] R3BROOT, <https://github.com/R3BRootGroup/R3BRoot/tree/dev/actaf>
- [2] <https://github.com/Dolezal-Patrik/GSI-FAIR---Summer-school---ACTAF-Data-analysis>